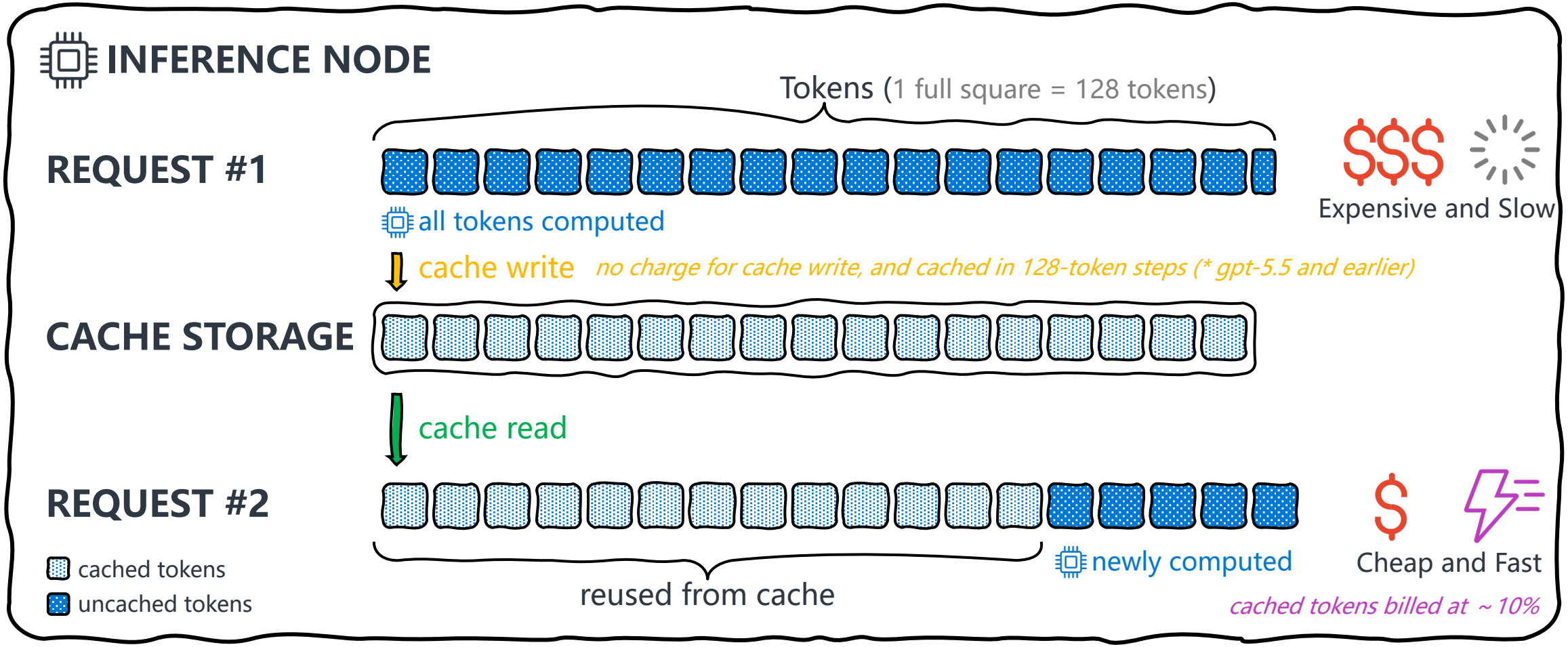
Microsoft

Azure Context Cache

Improving prompt cache hit rate on Microsoft Foundry

Jiqing You
Sr Solution Engineer, AI Apps

How Prompt Caching Works



What Determines a Cache Hit

minimum 1024 tokens

under 1024, never cached

1024

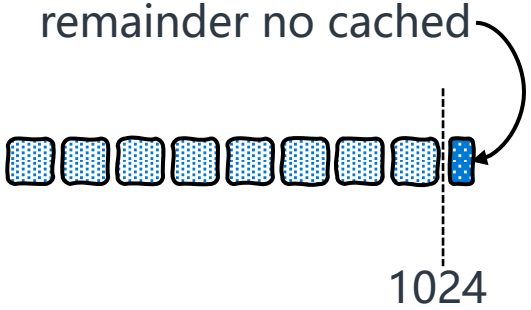
1024 or more, cached

cached tokens

uncached tokens

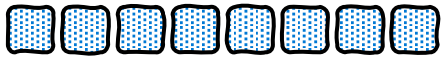
** gpt-5.5 and earlier*

128 tokens at a time



match from the start

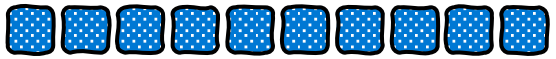
cached tokens



append at tail, still hits

change in the middle, everything after is lost

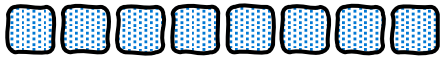
all tokens computed



** gpt-5.5 and earlier*

longest match wins

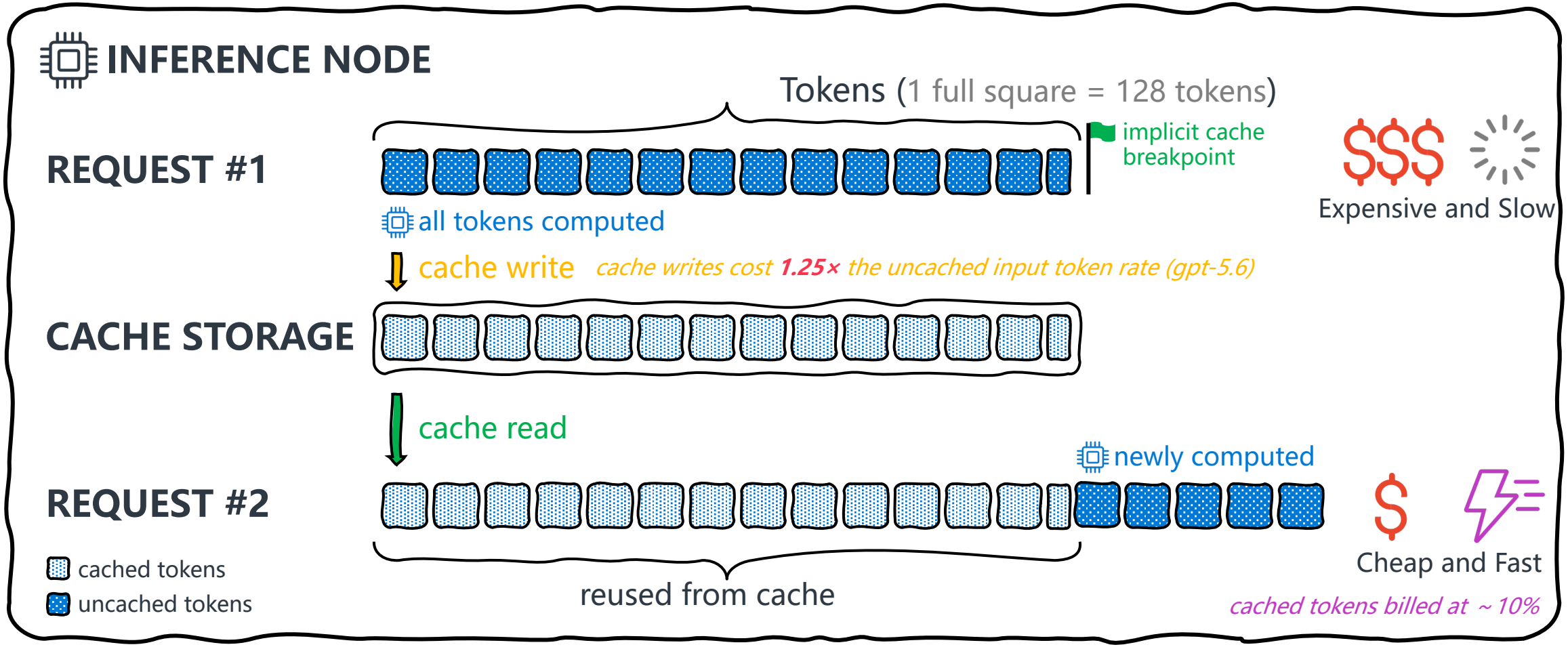
cache A



cache B

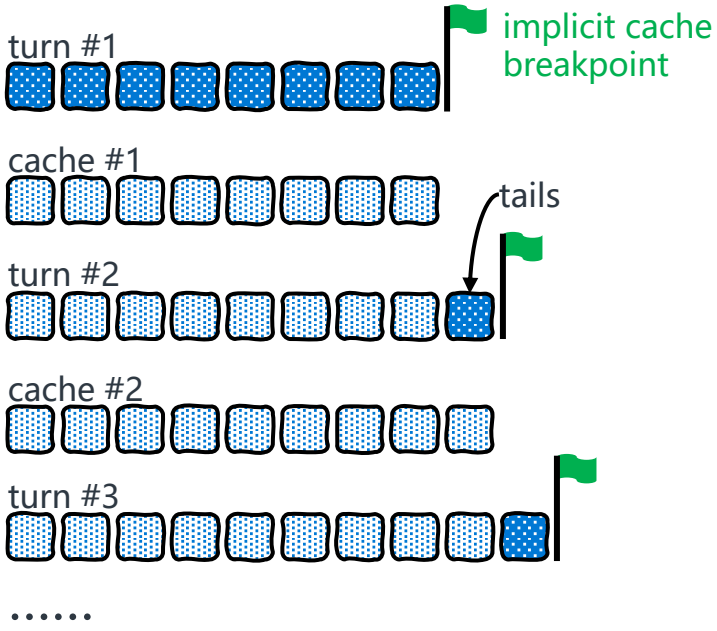
new request match B

What Changed in GPT-5.6



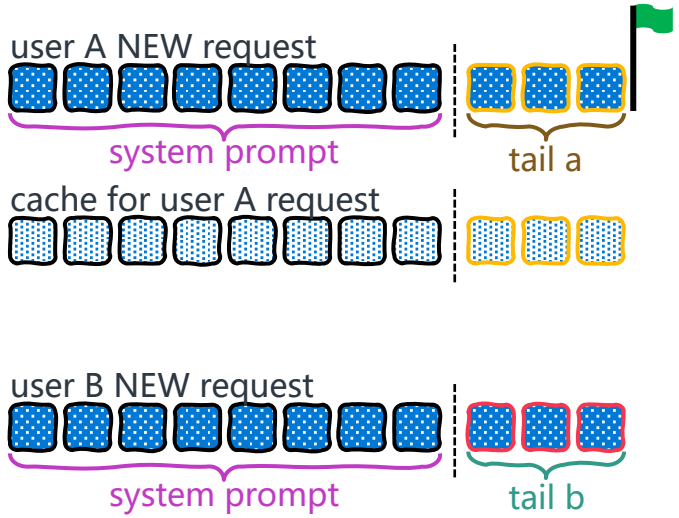
What Changed in GPT-5.6

same chat, many turns



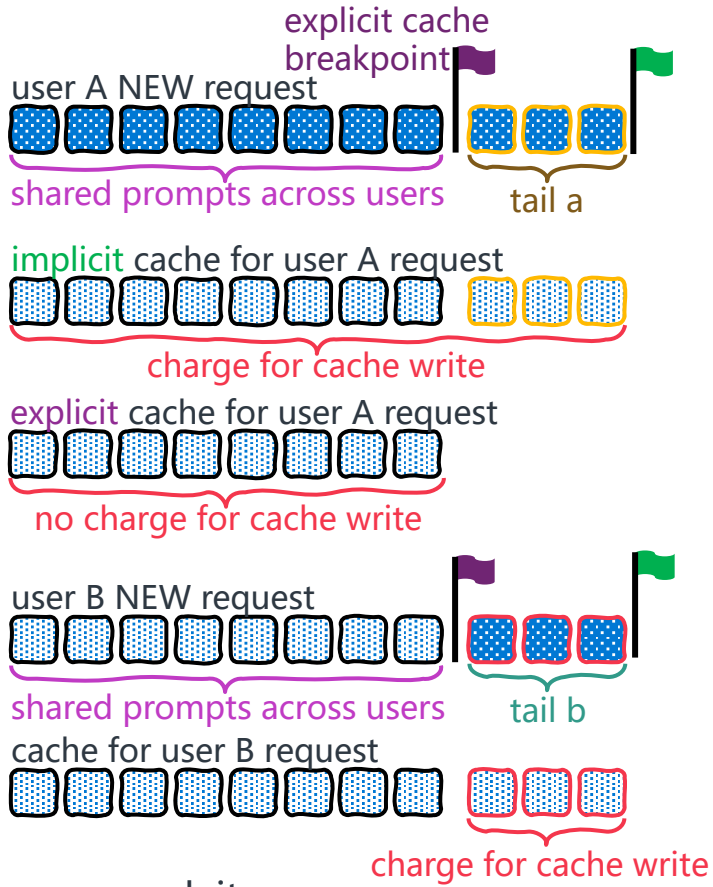
each turn reuses the turn before it

many users, one prompt/turn



different tails, the cut differs, no reuse, and you still pay to write

combine implicit and explicit



cached tokens

uncached tokens

IMPLICIT cuts at the latest message, EXPLICI cuts at where you mark it.
The overlap is not paid for twice, ONLY the new tail is written

Using Implicit and Explicit Cache Breakpoints

Prompt caching still worked without prompt_cache_options, preserving backward compatibility with GPT-5.5 and earlier models. However, GPT-5.6 reported cache_write_tokens only when prompt_cache_options was explicitly provided, for example, {"mode": "implicit"}.

```python
from openai import OpenAI

client = OpenAI(
    base_url="https://xxxx.openai.azure.com/openai/v1",
    api_key="xxxx",
)

# stable system prompt
system_prompts = ""

response = client.responses.create(
    model="gpt-5.6-sol",
    instructions=system_prompts,
    input="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
)

print(response.output_text)
print(response.usage.input_tokens_details.cache_write_tokens)
```

```python
from openai import OpenAI

client = OpenAI(
    base_url="https://xxxx.openai.azure.com/openai/v1",
    api_key="xxxx",
)

# stable system prompt
system_prompts = ""

response = client.responses.create(
    model="gpt-5.6-sol",
    instructions=system_prompts,
    prompt_cache_options={"mode": "implicit"},
    input="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
)

print(response.output_text)
print(response.usage.input_tokens_details.cache_write_tokens)
```

Using Implicit and Explicit Cache Breakpoints

```python
# stable system prompt
system_prompt = "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

response = client.responses.create(
    model="gpt-5.6-sol",
    prompt_cache_options={"mode": "implicit"},
    input=[
        {
            "role": "developer",
            "content": [
                {
                    "type": "input_text",
                    "text": system_prompt,
                    "prompt_cache_breakpoint": {"mode": "explicit"},
                }
            ],
        },
        {
            "role": "user",
            "content": [
                {
                    "type": "input_text",
                    "text": "xxxxxxxxxxxxxxxxxxxx",
                }
            ],
        },
    ],
)

print(response.output_text)
print(response.usage.input_tokens_details.cache_write_tokens)
```

#1 Write Only the Explicit Prefix (Explicit Mode)

```
{
  "input_tokens": 17339,
  "input_tokens_details": {
    "cache_write_tokens": 12888,
    "cached_tokens": 0
  }
}
```

#2 Hit the Explicit Prefix and Write the Extended Implicit Prefix (Implicit Mode)

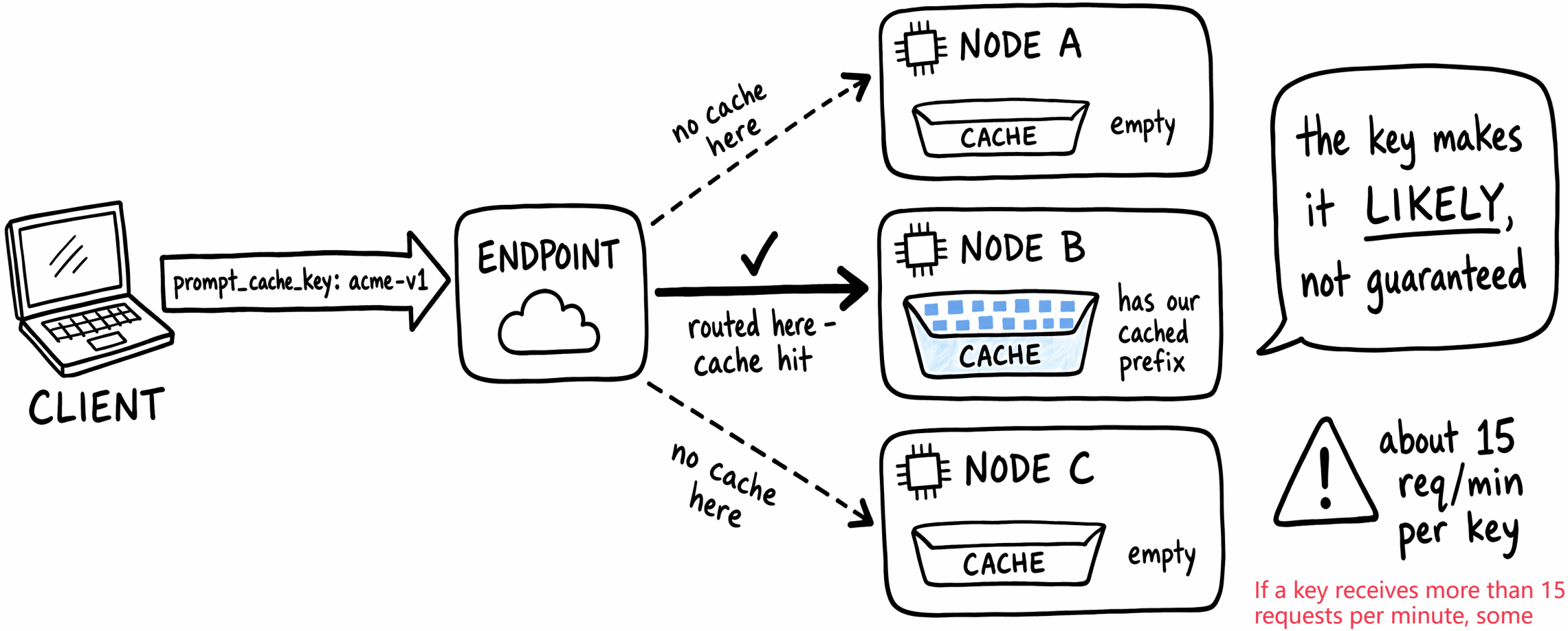
```
{
  "input_tokens": 17339,
  "input_tokens_details": {
    "cache_write_tokens": 4448,
    "cached_tokens": 12888
  }
}
```

12888+4448=17336

#3 Repeat Request 2 Unchanged to Verify the Extended Implicit Prefix Was Cached

```
{
  "input_tokens": 17339,
  "input_tokens_details": {
    "cache_write_tokens": 0,
    "cached_tokens": 17336
  }
}
```

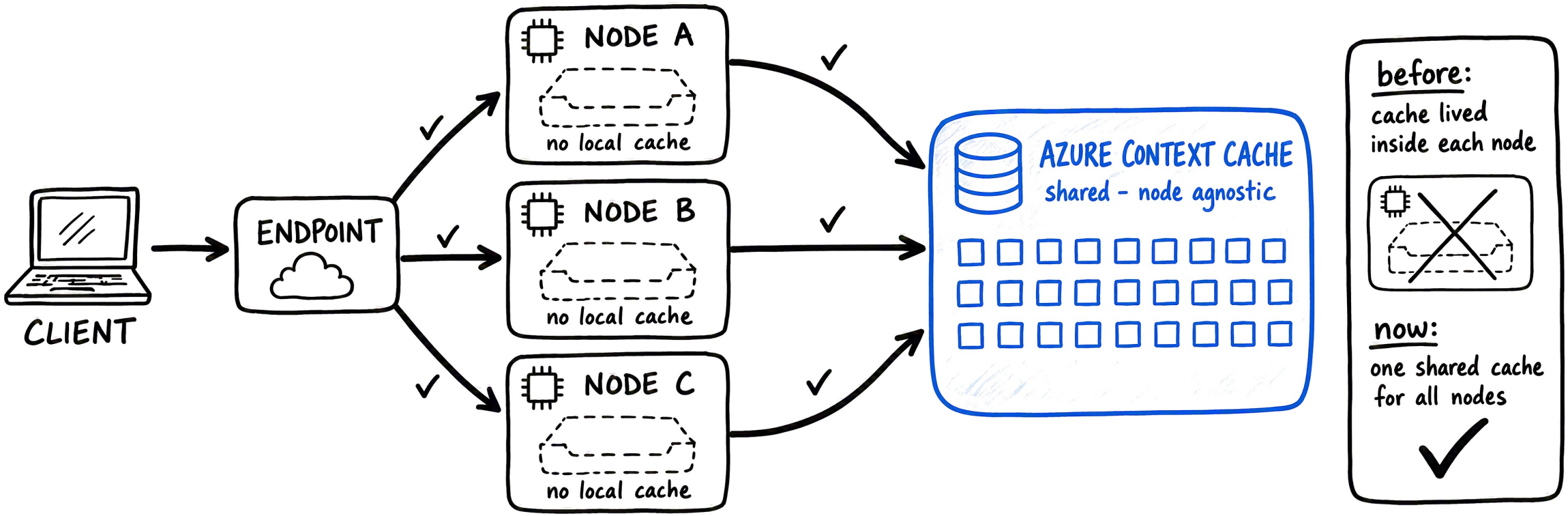
Improving Hit Rate with Cache Key



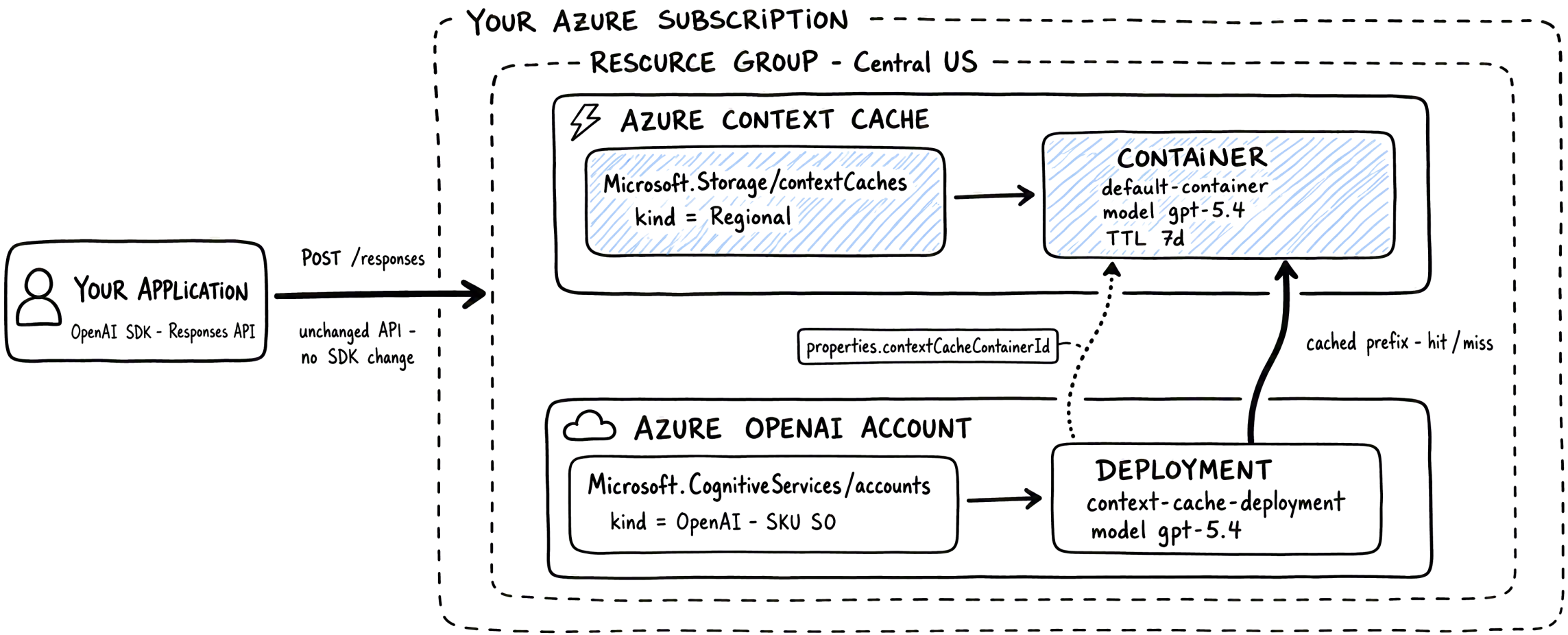
If a key receives more than 15 requests per minute, some requests may miss the cache.

Improving Hit Rate with Azure Context Cache

Azure Context Cache delivers durable, node-agnostic prompt caching for Microsoft Foundry AOAI deployments, guaranteeing cache reuse across concurrent users, long tool waits, multi-turn conversations, and any traffic pattern that defeats today's node-local caches.



Azure Context Cache Architecture



Limitations during Private Preview

- Request access to the private preview at aka.ms/foundry-explicit-caching-signup
- **Availability**
  - Region: Central US (recommended) and Sweden Central
  - Model: gpt-5.4, version 2026-03-05-contextcache, DataZone only
- **Usage Restrictions**
  - No load or stress testing, functional validation only
  - Not for production workloads
- **Pricing**
  - Free during private preview
  - Cache writes and cache storage will be billed starting at Public Preview — expected September 2026

Microsoft

Thank you

Jiqing You
Sr Solution Engineer, Cloud&AI Apps